

2.1

- **IF:** instruction fetch
- **ID:** instruction decode / register read
- **EX:** execute (ALU, address calculation)
- **MEM:** memory access
- **WB:** write back to registers

Between each stage is a pipeline register:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

These registers: hold all signals needed by the next stage are why you see “lots of wires passing through” are not logic, just storage

2.3

Malte hat ein Assembly-Programm geschrieben, um sich mit der Speicher- struktur von RISC-V vertraut zu machen:

```
.data
zahl1: .word 0x0ca7be5a
zahl2: .word 0xbaedfeed

.text
la t0 zahl1
addi t0 t0 2
pnmbokibvcy7sxx6rewq
```

Leider ist seine Katze über die Tastatur gelaufen und hat die Zeile 8 ver- unstaltet. Er weiß noch, dass er entweder ein Byte, Half word oder Word von der Adresse in t0 in t1 geladen hat und die Ausgabe in t1 am Ende 0xfeed0ca7 war.

a) Welche Ausgaben erhalten Sie jeweils für das Laden eines Bytes, Halb- wortes und Wortes? Wie ändert sich das Ergebnis im Allgemeinen für ein unsigniertes Laden? b) Wir ändern die Instruktion in Zeile 7 zu addi t0 t0 3. Wie ändert sich das Alignment im Vergleich zu vorher? c) Wie kann trotz ungültigen Alignment auf Speicher zugegriffen werden? Warum sollten Sie dies vermeiden?

Lösung

a) Byte: lb t1, 0 t0 -> fffffffa7 Half_Word: lh t1, 0 t0 -> 00000ca7 Word: lw t1, 0 t0 -> feed0ca7

Bei einem unsigniertem Laden wird der Wert als eine positive Zahl interpretiert. In unserem Szenario unterscheidet sich das Ergebnis nur beim Laden eines Bytes: lbu t1, 0 t0 ergibt 000000a7 statt fffffffa7.

b) Ausgehend von einer standardmäßigen Startadresse für .data (also word-aligned), ist das Alignment mit der Instruktion: addi t0, t0 2:

Byte: immer naturally aligned ("Adresse" % 1 = 0) Half Word: naturally aligned Word: not naturally aligned

Bei addi t0, t0 3 ist die Adresse auch für das half word nicht mehr naturally aligned.

c) Die CPU kann bei ungültigem Alignment durch wiederholtes Speichern und Laden konteragieren. Die genauen Techniken hängen von der Implementation ab (shifting, masking, merging). Solche

Operationen sind komplexer , langsamer (wiederholtes Lesen == mehr Zyklen) und evtl. nicht mehr atomar – es wird nicht mehr garantiert, dass sie “in einem Schritt” geschehen.

Aus dem Spec:

Loads and stores whose effective address is not naturally aligned to the referenced datatype (i.e., the effective address is not divisible by the size of the access in bytes) have behavior dependent on the EEI.

Even when misaligned loads and stores complete successfully, these accesses might run extremely slowly depending on the implementation (e.g., when implemented via an invisible trap). Furthermore, whereas naturally aligned loads and stores are guaranteed to execute atomically, misaligned loads and stores might not, and hence require additional synchronization to ensure atomicity.

– RISC-V Unprivileged ISA Spec ch. 2, 2.7. Load and Store Instructions